Telegram Assistant Builder Playbook

This document explains how the n8n Telegram Personal Assistant workflow was designed and how you would build a similar project from scratch.

It focuses on the practical questions:

- How do you choose which n8n nodes to use?
- How do you set up each node?
- How do you design the database schema?
- How do you connect all branches?
- What safety and maintenance factors matter?
- How would you explain this build to another engineer or employer?

The project uses:

- Telegram as the chat interface
- n8n as the workflow engine
- OpenAI as the parser, image reader, and voice transcriber
- Supabase Postgres as the database
- Git as the source of truth for workflow exports and scripts

---

1. The First Principle: Nodes Are Chosen By Responsibility

In n8n, a node is chosen because the workflow needs a specific responsibility.

Use this mental model:

```
Trigger node: starts the workflow
Code node: transforms or validates data
Switch node: chooses a branch
HTTP Request node: talks to an API or database
AI/API node: understands unstructured input
Telegram node: replies to the user
Database node/API: saves, reads, updates, or deletes data
```

The workflow is not a random set of boxes. It is a backend application drawn visually.

In normal code, the same project might look like this:

```
receiveWebhook()
normalizeInput()
dedupeUpdate()
routeCommand()
parseWithAI()
saveToDatabase()
sendTelegramReply()
```

In n8n, each of those steps becomes one or more nodes.

---

2. The Build Order

A project like this should be built in layers.

Do not start with 80 nodes. Start small.

Recommended build order:

```
1. Telegram receives a text message
2. Normalize the Telegram payload
3. Send the text to OpenAI
4. Parse OpenAI JSON
5. Save one expense to Supabase
6. Reply to Telegram
7. Add task/reminder routing
8. Add pending confirmation safety
9. Add voice notes
10. Add receipt photos
11. Add memory and parser context
12. Add budget commands
13. Add duplicate protection and retries
14. Add tests, export scripts, and deployment steps
```

This is how professional workflows grow: one working path first, then branches, then safety.

---

3. Important Setup Before Building Nodes

Before creating n8n nodes, prepare these things.

Telegram Setup

You need:

- A Telegram bot created through BotFather
- The bot token
- A private chat with the bot
- Your chat ID
- n8n Telegram credential using the bot token

Important Telegram decisions:

- Enable message updates
- Enable callback query updates if you use buttons
- Enable file download if you handle voice or photos
- Use dynamic chat ID replies instead of hard-coding only one ID

In this project, reply nodes use:

```
{{ $('Normalize Telegram Input').item.json.chatId || '5379148910' }}
```

The fallback chat ID is only safety. The real chat ID comes from the incoming Telegram update.

n8n Setup

You need:

- n8n running locally or on a server
- A public webhook URL
- ngrok or another tunnel if running locally
- Telegram credentials

- OpenAI credentials
- Supabase service role key available to n8n

Important environment variable:

```
SUPABASE_SERVICE_ROLE_KEY
```

The workflow uses this key in Supabase HTTP nodes.

OpenAI Setup

You need:

- OpenAI API key saved as an n8n credential
- A model for parsing text and images
- A transcription model for voice
- An embedding model for memories

In this project:

- Text and receipt parsing use the OpenAI Responses API
- Voice transcription uses OpenAI transcription
- Memory embeddings use OpenAI embeddings

Supabase Setup

You need:

- Supabase project URL
- Service role key
- Postgres schema and tables
- Public REST views for n8n
- RLS policies allowing service role access

Important principle:

The real tables live in the `assistant` schema. Public views expose them through Supabase REST.

Example:

```
assistant.expenses
public.assistant_expenses
```

n8n calls the public REST view:

```
/rest/v1/assistant_expenses
```

---

4. Database Schema Design

The database is designed around the assistant's business objects.

Ask:

```
What things does the assistant need to remember permanently?
```

For this assistant, the answer is:

- Expenses
- Tasks and reminders
- Logs
- Preferences
- Memories
- Budgets
- Pending confirmations
- Processed Telegram updates

Expenses Table

Purpose:

Store every saved expense or receipt.

Core columns:

```
create table assistant.expenses (
  id uuid primary key default gen_random_uuid(),
  expense_date date not null default current_date,
  merchant text,
  amount numeric(12, 2) not null check (amount >= 0),
  currency text not null default 'AED',
  category text not null default 'Other',
  payment_method text,
  card text,
  notes text,
  source text not null default 'telegram',
  confidence numeric(4, 3) check (confidence is null or (confidence >= 0 and confidence <= 1)),
  raw_payload jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now()
);
```

Why these fields:

- `expense_date` : lets the dashboard and budget checks group by date
- `merchant` : where money was spent
- `amount` : the required money value
- `currency` : defaults to AED
- `category` : Food, Groceries, Bills, and so on
- `payment_method` : cash, card, Apple Pay, etc.
- `card` : exact card name such as ENBD Credit
- `source` : text, receipt, voice, confirmed pending
- `confidence` : how confident OpenAI was
- `raw_payload` : saves the original parsed JSON for debugging

Indexes:

```
create index expenses_date_idx on assistant.expenses (expense_date desc);
create index expenses_category_idx on assistant.expenses (category);
create index expenses_card_idx on assistant.expenses (card) where card is not null;
```

Why indexes:

- Dashboard queries often sort by date

- Budget checks filter by category
- Card summaries filter by card

Tasks Table

Purpose:

Store todos and reminders.

Core columns:

```
create table assistant.tasks (
  id uuid primary key default gen_random_uuid(),
  task text not null,
  type text not null default 'todo' check (type in ('todo', 'reminder')),
  status text not null default 'open' check (status in ('open', 'sent', 'done', 'cancelled')),
  priority text not null default 'normal' check (priority in ('low', 'normal', 'high')),
  due_at timestamptz,
  notes text,
  raw_payload jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  completed_at timestamptz
);
```

Why these fields:

- `type` : separates todo from reminder
- `status` : tracks lifecycle
- `due_at` : needed for reminders
- `completed_at` : needed for reports and history

Logs Table

Purpose:

Record what happened during workflow processing.

This is useful for debugging and auditing.

Core columns:

```
create table assistant.logs (
  id bigserial primary key,
  workflow text not null,
  chat_id text,
  raw_input text,
  intent text,
  parsed_json jsonb,
  status text not null,
  message text,
  missing_fields text[],
  execution_source text,
  created_at timestamptz not null default now()
);
```

Why logs matter:

If the bot gives a strange response, logs help answer:

```
What did the user send?
What did OpenAI parse?
What intent was chosen?
Was anything missing?
```

Preferences Table

Purpose:

Store global assistant settings.

Example preferences:

```
timezone = Asia/Dubai
default_currency = AED
categories = Food, Transport, Groceries, ...
parser_context = category defaults and card aliases
```

Schema:

```
create table assistant.preferences (
  key text primary key,
  value jsonb not null,
  updated_at timestamptz not null default now()
);
```

Why JSONB:

Preferences can have different shapes. Some are strings, some are arrays, some are objects.

Memories Table

Purpose:

Store user-specific facts the assistant should remember.

Schema:

```
create table assistant.memories (
  id uuid primary key default gen_random_uuid(),
  memory_type text not null,
  content text not null,
  metadata jsonb not null default '{}'::jsonb,
  embedding vector(1536),
  created_at timestamptz not null default now()
);
```

Why vector:

Embeddings make it possible to search memories by meaning later.

Example memory:

```
Costa is Food
```

Budgets Table

Purpose:

Store category spending limits.

Schema:

```
create table assistant.budgets (
  id uuid primary key default gen_random_uuid(),
  category text not null check (length(trim(category)) > 0),
  amount numeric(12, 2) not null check (amount > 0),
  currency text not null default 'AED',
  period text not null default 'monthly' check (period in ('monthly', 'weekly')),
  active boolean not null default true,
  notes text,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique (category, period)
);
```

Why unique category and period:

The user should not have two active monthly Groceries budgets. Upsert works cleanly when `category, period` is unique.

Pending Actions Table

Purpose:

Store actions that need user confirmation.

This is used when the assistant is uncertain or the action is important.

Expected fields:

```
id
chat_id
action_type
status
payload
source
message_id
expires_at
created_at
updated_at
```

Why this matters:

If the user says:

```
maybe spent around 20 at Costa
```

The workflow should not blindly save. It saves a pending action and asks:

```
Please confirm before I save this.
```

Then `confirm` saves it, and `cancel` discards it.

Processed Telegram Updates Table

Purpose:

Prevent duplicate processing.

Schema:

```
create table assistant.processed_telegram_updates (
  telegram_update_id bigint primary key,
  chat_id text,
  update_type text,
  received_at timestamptz,
  processed_at timestamptz not null default now(),
  raw_payload jsonb not null default '{}'::jsonb
);
```

Why primary key:

Telegram `update_id` is unique. Making it the primary key lets Supabase reject duplicates.

Public REST Views

n8n calls Supabase REST through public views:

```
create or replace view public.assistant_expenses as
select * from assistant.expenses;
```

Why views:

- Keep real tables organized in the `assistant` schema
- Give n8n stable REST names
- Avoid exposing internal schema details directly

RLS and Service Role

Tables have row-level security enabled.

The workflow uses service role policies:

```
create policy "service role manages expenses"
on assistant.expenses
for all
to service_role
using (true)
with check (true);
```

Important:

The service role key is powerful. It should be used only server-side, inside n8n or trusted backend code. Do not expose it in a browser.

---

5. Choosing And Setting Up Each Node Type

This section explains the main node types and why they were chosen.

---

6. Telegram Trigger Node

Node:

```
Telegram Inbox
```

Node type:

```
n8n-nodes-base.telegramTrigger
```

Why chosen:

The workflow starts when Telegram sends an update.

Important settings:

```
Updates: message, callback_query
Download files: true
Image size: large
Credential: Telegram account
```

Why `message`:

Text, voice, and photo messages are Telegram messages.

Why `callback_query`:

Inline button clicks arrive as callback queries.

Why download files:

Voice notes and receipt photos need binary data.

How to set it up:

```
1. Add Telegram Trigger node.
2. Select Telegram credential.
3. Choose message updates.
4. Add callback_query updates if using buttons.
5. Turn on download files.
6. For images, choose a useful image size such as large.
7. Activate workflow so Telegram receives the webhook URL.
```

Common mistake:

If files are not downloaded, later photo or voice nodes will not have binary data.

---

7. Normalize Telegram Input Code Node

Node:

```
Normalize Telegram Input
```

Node type:

```
n8n-nodes-base.code
```

Why chosen:

Telegram payloads are inconsistent. A Code node lets us convert everything into one stable shape.

This node extracts:

```
chatId
telegramUpdateId
originalText
source
needsTranscription
pendingCommand
pendingActionId
memoryCommand
budgetCommand
isCallback
callbackQueryId
mediaMimeType
binaryKey
voiceFileId
openaiBody
```

How to think about this node:

It is the workflow's controller input adapter.

The rest of the workflow should not care whether Telegram sent:

- `message.text`
- `message.caption`
- `message.voice`
- `message.photo`
- `callback_query.data`

After normalization, the rest of the workflow reads consistent fields.

Important logic inside:

```
If callback query exists, use callback data.
Else use message text or caption.
If voice exists, mark needsTranscription true.
If photo exists, mark source telegram_photo.
If text matches confirm/cancel/undo/memory/budget, set pendingCommand.
Build the initial OpenAI request body.
```

Why parse budget commands here:

Budget commands are simple and deterministic. They do not need AI.

Example:

```
groceries 2k
```

The Code node can parse that faster and more safely than OpenAI.

Important setup practice:

Keep this node's output clean. If you add new branches later, add one new normalized key instead of making every downstream node inspect raw Telegram JSON.

---

8. Dedupe Nodes

Nodes:

```
Check Telegram Update Dedupe
Route Telegram Update Dedupe
Mark Telegram Update Processed
Reply Duplicate Telegram Update
```

Why chosen:

Telegram may retry delivery. Workflows that save money records must be idempotent.

Check Telegram Update Dedupe

Node type:

```
HTTP Request
```

Method:

```
POST
```

URL pattern:

```
https://SUPABASE_URL/rest/v1/assistant_processed_telegram_updates?on_conflict=telegram_update_id
```

Headers:

```
apikey: {{ $env.SUPABASE_SERVICE_ROLE_KEY }}
Authorization: {{ 'Bearer ' + $env.SUPABASE_SERVICE_ROLE_KEY }}
Content-Type: application/json
Prefer: resolution=ignore-duplicates,return=representation
```

Body:

```
{
  "telegram_update_id": "...",
  "chat_id": "...",
  "update_type": "...",
  "received_at": "...",
  "raw_payload": {}
}
```

Why this works:

If the update is new, Supabase inserts it and returns a row. If it is a duplicate, Supabase ignores it and returns no inserted row.

Route Telegram Update Dedupe

Node type:

```
Switch
```

Logic:

```
If insert returned telegram_update_id -> continue
Else -> duplicate reply
```

Mark Telegram Update Processed

Node type:

```
Code
```

Why chosen:

The dedupe HTTP node's output is the Supabase inserted row. But the workflow needs the original Telegram normalized object to continue.

This Code node restores the original normalized item and keeps binary data.

---

9. Switch Nodes

Switch nodes are decision points.

They are chosen when the workflow needs routing.

Route Pending Command

Purpose:

Send direct commands to direct branches.

Outputs:

```
0 confirm
1 cancel
2 undo expense
3 remember memory
4 forget memory
5 recall memory
6 set budget
7 delete budget
8 list budgets
9 normal text/photo/voice path
```

Why this matters:

Commands like `list budgets` do not need OpenAI. Routing them directly is faster and safer.

Route Voice

Purpose:

Separate voice from non-voice.

Logic:

```
needsTranscription ? voice branch : photo/text branch
```

Route Intent

Purpose:

After OpenAI parsing, decide what business action to take.

Outputs:

```
0 expense or receipt
1 todo or reminder
2 task_done
3 unclear or invalid
```

Important:

If the parse is missing fields or requires confirmation, it goes to the safe branch instead of saving directly.

---

10. OpenAI Parse Message Node

Node:

```
OpenAI Parse Message
```

Node type:

```
HTTP Request
```

Why HTTP Request:

The workflow uses the OpenAI Responses API directly with a custom JSON body.

Method:

```
POST
```

URL:

```
https://api.openai.com/v1/responses
```

Authentication:

```
OpenAI n8n credential
```

Body:

```
{{ JSON.stringify($json.openaiBody) }}
```

The `openaiBody` is built earlier in `Normalize Telegram Input`, then enriched by `Apply Memory Context`.

Why build the body earlier:

It lets text, voice, and photo paths all converge into the same OpenAI node.

---

11. Building The OpenAI JSON Schema

The parser asks OpenAI to return strict JSON.

The schema includes:

```
intent
date
merchant
amount
currency
category
payment_method
card
notes
task
type
priority
due_at
task_match_text
missing_fields
confidence
```

Why strict schema:

The workflow needs predictable keys. If OpenAI returns free text, downstream nodes become fragile.

Intent enum:

```
expense
receipt
todo
reminder
task_done
daily_summary
weekly_summary
unknown
```

Category enum:

```
Food
Transport
Groceries
Bills
Shopping
Business
Health
Entertainment
Travel
Trading
Other
```

Important schema design rules:

```
1. Include every field downstream nodes need.
2. Give fallback fields empty strings or arrays.
3. Use enums for values that must stay consistent.
4. Include confidence for safety decisions.
5. Include missing_fields so the workflow can ask for clarification.
```

Example parsed output:

```
{
  "intent": "expense",
  "date": "2026-05-15",
  "merchant": "Costa",
  "amount": 25,
  "currency": "AED",
  "category": "Food",
  "payment_method": "card",
  "card": "ENBD Visa",
  "notes": "",
  "task": "",
  "type": "",
  "priority": "",
  "due_at": "",
  "task_match_text": "",
  "missing_fields": [],
  "confidence": 0.93
}
```

---

12. Parse OpenAI Result Code Node

Node:

```
Parse OpenAI Result
```

Why chosen:

OpenAI's API response contains nested output. The workflow needs clean JSON.

This node:

```
1. Finds output_text from OpenAI response.
2. Parses JSON safely.
3. Adds fallback values.
4. Decides if the result is valid.
5. Builds a human-readable confirmation.
6. Carries original chat and source data forward.
```

Important safety logic:

If JSON parsing fails:

```
intent = unknown
missing_fields = ["intent"]
confidence = low
```

Then the workflow routes to clarification instead of saving bad data.

---

13. Supabase HTTP Nodes

Supabase is accessed through HTTP Request nodes.

Why HTTP instead of a native database node:

Supabase REST is stable, simple, and works well with service role headers.

Common headers:

```
apikey: {{ $env.SUPABASE_SERVICE_ROLE_KEY }}
Authorization: {{ 'Bearer ' + $env.SUPABASE_SERVICE_ROLE_KEY }}
Content-Type: application/json
Prefer: return=representation
```

Common methods:

```
GET: read rows
POST: insert rows
PATCH: update rows
DELETE: delete rows
```

Append Expense

Method:

```
POST
```

URL:

```
/rest/v1/assistant_expenses
```

Body:

```
{{ JSON.stringify($json) }}
```

Input comes from `Prepare Expense Row`.

Read Budget After Expense

Method:

```
GET
```

Purpose:

Find active monthly budget for the expense category.

URL concept:

```
/rest/v1/assistant_budgets
  ?select=category,amount,currency,period,active
  &active=eq.true
  &period=eq.monthly
  &limit=1
  &category=eq.CATEGORY
```

Read Monthly Spend After Expense

Method:

```
GET
```

Purpose:

Read all expense amounts for the same category in the current month.

Then the Code node sums them.

---

14. Telegram Reply Nodes

Reply nodes are chosen whenever the user needs feedback.

Examples:

```
Confirm Expense
Confirm Task
Confirm Done
Confirm Unknown
Confirm Pending Request
Confirm Budget Command
Confirm Memory Command
Confirm Voice Problem
Reply Duplicate Telegram Update
```

Important settings:

```
Chat ID: {{ $('Normalize Telegram Input').item.json.chatId || '5379148910' }}
Text: dynamic expression from current JSON
Append attribution: false
Retry on fail: true
Max tries: 3
Wait between tries: 2000ms
```

Why retries:

Telegram API calls can fail briefly. Retrying prevents missed confirmations.

Why dynamic chat ID:

The bot replies to the chat that sent the message.

---

15. Receipt Photo Branch

Receipt photos need special handling.

Flow:

```
Telegram Inbox
Normalize Telegram Input
Route Voice
Prepare Photo Image
Read Parser Context
Read Recent Memories
Apply Memory Context
OpenAI Parse Message
```

Prepare Photo Image

Why chosen:

n8n stores downloaded photos as binary files. OpenAI needs actual image data.

This node:

```
1. Reads the binary key from normalized input.
2. Loads the file buffer through n8n helpers.
3. Converts it to base64.
4. Creates ...
5. Replaces the OpenAI image_url.
```

Important lesson:

When multiple nodes mutate a shared object like `openaiBody`, later nodes must start from the newest version. Otherwise a later node can accidentally undo an earlier preparation step.

That exact issue was fixed in this project.

---

16. Voice Branch

Voice needs a separate path because it must become text first.

Flow:

```
Route Voice
Download Voice
Prepare Voice File
Route Voice File
Transcribe Voice
Apply Voice Transcript
Route Voice Transcript
Read Parser Context
OpenAI Parse Message
```

Node responsibilities:

```
Download Voice: gets audio file from Telegram
Prepare Voice File: formats it for OpenAI transcription
Transcribe Voice: calls OpenAI transcription
Apply Voice Transcript: replaces original text with transcript
Route Voice Transcript: continues or sends error reply
```

Important factor:

Always have a failure reply for voice. If transcription fails, the user should know.

---

17. Pending Confirmation Branch

Purpose:

Avoid saving uncertain or risky actions automatically.

Flow for uncertain parse:

```
Route Intent
Route Clarify or Pending
Prepare Pending Action
Append Pending Action
Confirm Pending Request
```

Flow for confirm:

```
Route Pending Command
Read Pending for Confirm
Prepare Confirmed Pending
Route Confirmed Pending
Append Confirmed Expense or Task
Mark Pending Confirmed
Confirm Pending Command
```

Flow for cancel:

```
Route Pending Command
Read Pending for Cancel
Prepare Cancel Pending
Mark Pending Cancelled
Confirm Pending Command
```

Important safety rules:

```
If callback data includes a pending action ID, target that exact ID.
If user types confirm/cancel, only target the latest pending item for same chat ID.
Only target status pending.
Only target items where expires_at has not passed.
```

Why this matters:

Without these filters, the assistant might confirm an old or unrelated action.

---

18. Budget Branch

Budget commands are parsed in `Normalize Telegram Input`.

Supported input:

```
groceries 2k
food limit 300 weekly
set monthly dining cap 1500
entertainment 1500aed
budget food 2,000
list budgets
```

Set budget flow:

```
Route Pending Command
Prepare Budget Upsert
Upsert Budget
Confirm Budget Command
```

List budget flow:

```
Route Pending Command
Read Budgets
Format Budget List
Confirm Budget Command
```

Delete budget flow:

```
Route Pending Command
Prepare Budget Delete
Delete Budget
Confirm Budget Command
```

Important parsing rules:

```
k means thousand
2k becomes 2000
2,000 becomes 2000
1500aed becomes 1500
weekly/monthly becomes period
default period is monthly
category text is preserved after cleanup
```

---

19. Budget Warning Branch

After an expense is saved, the workflow checks budget progress.

Flow:

```
Append Expense
Read Budget After Expense
Read Monthly Spend After Expense
Append Budget Warning
Confirm Expense
```

Logic:

```
monthly spend / monthly budget = percentage
if percentage >= 100%, append over-budget warning
else if percentage >= 80%, append budget warning
else normal confirmation
```

Important:

The warning does not block saving. It only informs the user.

---

20. Memory Branch

Memory commands let the assistant store user preferences.

Examples:

```
remember Costa is Food
show memories
forget Costa
```

Create memory flow:

```
Prepare Memory Embedding
Create Memory Embedding
Prepare Create Memory
Create Memory
Confirm Memory Command
```

Recall flow:

```
Read Memories for Recall
Prepare Memory Recall
Confirm Memory Command
```

Forget flow:

```
Read Memories for Forget
Prepare Forget Memory
Route Forget Memory
Forget Memory
Confirm Memory Command
```

Parser context flow:

```
Read Parser Context
Read Recent Memories
Apply Memory Context
OpenAI Parse Message
```

Why memory context is added before OpenAI:

OpenAI can parse better if it knows:

```
Costa -> Food
Carrefour -> Groceries
ADCB Visa -> ADCB Credit
Default currency -> AED
```

---

21. How To Build This Workflow Step By Step In n8n

This is the practical build recipe.

Step 1: Create Supabase Schema

Create migrations for:

```
assistant schema
expenses table
tasks table
logs table
preferences table
memories table
budgets table
pending actions table
processed telegram updates table
public REST views
RLS policies and grants
```

Run migrations before building n8n nodes that depend on them.

Step 2: Add Credentials

In n8n, create:

```
Telegram credential
OpenAI credential
Supabase environment variable SUPABASE_SERVICE_ROLE_KEY
```

For Supabase HTTP nodes, you do not need a dedicated n8n Supabase credential if using REST with headers.

Step 3: Add Telegram Trigger

Create `Telegram Inbox`.

Configure:

```
Updates: message, callback_query
Download files: true
Image size: large
```

Run one test message and inspect the raw Telegram JSON.

Step 4: Add Normalize Code Node

Create `Normalize Telegram Input`.

Output a clean object:

```
{
  "chatId": "...",
  "telegramUpdateId": 123,
  "source": "telegram_text",
  "originalText": "...",
  "pendingCommand": "",
  "needsTranscription": false,
  "openaiBody": {}
}
```

Test with:

```
normal text
photo
voice
confirm
list budgets
```

Step 5: Add Dedupe

Create:

```
Check Telegram Update Dedupe
Route Telegram Update Dedupe
Mark Telegram Update Processed
Reply Duplicate Telegram Update
```

Test by replaying the same update or checking that repeated update IDs do not create duplicate rows.

Step 6: Add Command Router

Create `Route Pending Command`.

Start with fewer outputs if building from scratch:

```
0 confirm
1 cancel
2 normal
```

Add more outputs later:

```
undo
memory
budget
```

Step 7: Add OpenAI Text Parse Path

Create:

```
OpenAI Parse Message
Parse OpenAI Result
Route Intent
```

Start with only expense support.

Make sure OpenAI returns strict JSON.

Step 8: Add Expense Save

Create:

```
Prepare Expense Row
Append Expense
Confirm Expense
```

Test:

```
spent 25 aed at Costa with ENBD Visa
```

Expected:

```
One expense row in Supabase
One Telegram confirmation
```

Step 9: Add Tasks

Create:

```
Prepare Task Row
Append Task
Confirm Task
Read Tasks for Done
Find Task to Complete
Update Completed Task
Confirm Done
```

Test:

```
todo buy printer ink
done buy printer ink
```

Step 10: Add Pending Confirmation

Create pending-action table first.

Then create:

```
Route Clarify or Pending
Prepare Pending Action
Append Pending Action
Confirm Pending Request
Read Pending for Confirm
Read Pending for Cancel
Prepare Confirmed Pending
Prepare Cancel Pending
Mark Pending Confirmed
Mark Pending Cancelled
Confirm Pending Command
```

Test:

```
maybe spent around 20 at Costa
confirm
```

Step 11: Add Voice

Create:

```
Route Voice
Download Voice
Prepare Voice File
Transcribe Voice
Apply Voice Transcript
Route Voice Transcript
Confirm Voice Problem
```

After transcription, route back into the normal OpenAI parse path.

Step 12: Add Receipt Photos

Create:

```
Prepare Photo Image
```

Route photo messages through this node before OpenAI.

Test with a real Telegram receipt photo.

Check that OpenAI receives:

```
...
```

not:

```
filesystem-v2
```

Step 13: Add Memory Context

Create:

```
Read Parser Context
Read Recent Memories
Apply Memory Context
```

Place these before OpenAI parse.

Important:

For voice, use the transcribed payload. For photos, use the prepared photo payload. For text, use the normalized payload.

Step 14: Add Budget Commands

Create:

```
Prepare Budget Upsert
Upsert Budget
Prepare Budget Delete
Delete Budget
Read Budgets
Format Budget List
Confirm Budget Command
```

Test:

```
groceries 2k
list budgets
spent 1900 on groceries
```

Step 15: Add Budget Warning

After `Append Expense`, insert:

```
Read Budget After Expense
Read Monthly Spend After Expense
Append Budget Warning
Confirm Expense
```

Do the same after confirmed pending expenses.

Step 16: Add Logging

Create:

```
Prepare Log Row
Append Log
```

Attach it after parsing so every message can be inspected later.

Step 17: Add Tests And Export Script

Keep workflow JSON in Git.

Use a script like:

```
scripts/harden-active-inbox-workflow.mjs
```

Use tests to check important wiring:

```
Reply nodes use dynamic chat ID
Deduplication is early
Switch output count is correct
Photo path goes through Prepare Photo Image
Budget warning branch exists
Invalid parses clarify instead of saving
```

---

22. Important Factors Needed For A Good Workflow

Data Shape

Every node should know what fields it receives.

That is why normalization is critical.

Idempotency

If the same input arrives twice, the workflow should not create duplicate expenses.

That is why `telegramUpdateId` dedupe exists.

Safety

Do not auto-save uncertain actions.

Use pending confirmation when:

```
confidence is low
amount is missing
task text is missing
due date is unclear
user says maybe/around
```

Observability

You need logs and execution history.

When something fails, inspect:

```
latest n8n execution
failed node
input data
output data
HTTP response
Telegram webhook info
Supabase row
```

User Feedback

Every branch should reply.

No reply feels like the bot is broken, even if the backend did something.

Minimal AI Use

Use AI for messy human language.

Use code for deterministic commands.

Good AI use:

```
spent maybe 20 at costa
receipt image
voice transcript meaning
reminder phrasing
```

Good code use:

```
confirm
cancel
undo
list budgets
groceries 2k
```

Stable Names

Do not casually rename:

```
Supabase tables
REST views
credentials
environment variables
n8n node names used in expressions
```

n8n expressions often reference nodes by name:

```
$('Normalize Telegram Input').item.json.chatId
```

If you rename the node, expressions can break.

---

23. Maintenance Checklist

Before changing the workflow:

```
1. Export or inspect the current workflow.
2. Check git status.
3. Understand which branch you are touching.
4. Preserve existing node IDs where possible.
5. Make the smallest safe change.
6. Regenerate workflow exports if using scripts.
7. Validate JSON.
8. Run tests.
9. Import workflow to n8n.
10. Publish workflow.
11. Restart n8n if required.
12. Verify active workflows.
13. Verify Telegram webhook health.
14. Run live Telegram smoke test.
15. Commit and push.
```

Useful commands:

```
node scripts\harden-active-inbox-workflow.mjs
npm test -- --run
n8n import:workflow --input=workflows\telegram-assistant-inbox-budget-commands.json
n8n publish:workflow --id=telegram-personal-assistant-inbox-budget-commands
.\start-n8n-with-ngrok.ps1
git status
git add ...
git commit -m "message"
git push origin main
```

---

24. How To Debug A Broken Branch

Use this method:

```
1. Identify which user action failed.
2. Find the latest n8n execution.
3. Locate the first failed node.
4. Inspect that node's input and output.
5. Trace backward until the data first becomes wrong.
6. Fix the source of wrong data.
7. Add a test if possible.
8. Redeploy and smoke test.
```

Example from this project:

Problem:

```
Receipt photo sent, but no useful reply.
```

Evidence:

```
OpenAI received image_url = -v2
```

Trace:

```
Prepare Photo Image created valid base64.
Apply Memory Context rebuilt openaiBody from older normalized data.
That restored filesystem-v2.
```

Fix:

```
For photo messages, Apply Memory Context now starts from Prepare Photo Image output.
```

Lesson:

```
Do not patch blindly. Trace where the data becomes wrong.
```

---

25. How To Explain Node Choice In An Interview

Use this explanation:

```
I chose nodes based on responsibility. Telegram Trigger starts the workflow. A Code node normalizes the different
Telegram payload shapes. A Supabase HTTP node deduplicates Telegram update IDs. Switch nodes route commands, media,
and parsed intents. OpenAI is used only where the input is unstructured, such as text expenses, voice transcripts,
and receipt images. Supabase HTTP nodes save and read persistent data. Telegram nodes send clear replies. Safety
branches handle uncertainty through pending confirmations, and tests validate important wiring in the exported
workflow JSON.
```

If asked how you designed the schema:

```
I started from business objects: expenses, tasks, memories, budgets, pending confirmations, preferences, and
processed Telegram updates. Each table has constraints for data quality, indexes for common queries, RLS enabled,
and service-role policies because n8n acts as a trusted backend. Public views expose stable REST endpoints while
keeping real tables in the assistant schema.
```

If asked how you would maintain it:

```
I would preserve node IDs and table names, make targeted changes through scripts, regenerate workflow exports, run
tests, import and publish to n8n, restart if required, verify webhook health, and run live smoke tests for any
changed branch.
```

---

26. Simple Version You Can Build For Practice

If you want to practice, build this smaller version first:

```
Telegram Trigger
Normalize Code Node
OpenAI Parse HTTP Node
Parse Result Code Node
Prepare Expense Code Node
Supabase Insert HTTP Node
Telegram Reply Node
```

Only after that works, add:

```
Switch for task vs expense
Dedupe
Pending confirmation
Voice
Photo
Budgets
Memory
```

This is how you avoid being overwhelmed.

---

27. The Big Picture

The workflow is a backend system with a chat interface.

The architecture is:

```
Input layer:
Telegram Inbox

Normalization layer:
Normalize Telegram Input

Safety layer:
Dedupe, pending confirmation, retries

Routing layer:
Switch nodes

AI layer:
OpenAI parse, transcription, embeddings

Persistence layer:
Supabase REST nodes

Response layer:
Telegram reply nodes

Maintenance layer:
Workflow exports, scripts, tests, git
```

Once you see it this way, building and maintaining n8n workflows becomes much easier.